

Architecture Specification

Multi-Agent Orchestration: Claude-CLI & Antigravity (agy) Daemon Bridge

This document details the architectural topology, communication protocols, and execution guardrails for chaining `claude-cli` (acting as the master directing agent) to a background instance of the Google Antigravity CLI (`agy`). The architecture ensures seamless execution across local Windows development environments and headless Linux production servers, completely sidestepping OS-specific IPC complications and TTY shell limitations.

1. System Topology

To avoid terminal-scraping instability when running over standard programmatic subprocesses, the bridge is constructed as a **Python-based Model Context Protocol (MCP) Server** utilizing the official `google-antigravity` Python SDK.

- **Master Agent (Claude):** Manages the overarching project logic and user interaction. Calls the `delegate_to_antigravity` MCP tool when deep file editing or complex multi-step compilation tasks are required.
- **MCP Middleware:** A stateful Python process that translates tool schemas, enforces timeouts, handles token telemetry, and initializes the daemonized agent configuration.
- **Sub-Agent (Antigravity):** Operates as a persistent, background execution engine scoped strictly to the target workspace.

2. Communication & Payload Structure

The bridge operates on a "Delegated Task" pattern. The master agent passes a focused prompt and target directory, and the MCP server blocks until the sub-agent returns a deterministic JSON schema. Leveraging Python and Pydantic for the bridge perfectly balances execution speed with dynamic schema generation, while cleanly interacting with the target workspace (whether the codebase contains TypeScript, Rust, or Python).

Headless Execution Parity

Because the implementation uses the native SDK rather than shelling out to the binary with a forced pseudo-terminal (PTY), the daemon architecture fully supports remote invocation over SSH, requiring no interactive TTY layer.

Standard JSON Response Payload

The sub-agent is instructed to return the following structure upon task completion, allowing `claude-cli` to instantly parse results without reading through verbose reasoning loops:

```
{
  "status": "completed",
  "task_summary": "Refactored database connection logic and updated tests.",
  "files_modified": [
    {
      "path": "src/db.ts",
      "change_type": "modified",
      "diff_summary": "Replaced standard pool with optimized worker pool."
    }
  ],
  "terminal_outputs": [
    {
      "command": "npm run test",
      "exit_code": 0,
      "output": "PASS: src/db.test.ts"
    }
  ]
}
```

3. Sub-Agent Governance: SKILL.md

Rather than bloating the MCP server code with massive system prompts, the sub-agent's behavior is rigidly controlled via an Antigravity Skill. This file (`.agent/skills/claude_subagent/SKILL.md`) is dynamically loaded by the Python SDK upon initialization.

```
---
name: claude_subagent_protocol
description: Rules for executing technical tasks delegated by a master agent.
---
### Goal
Execute the assigned codebase modification safely and structure the final report into the strict JSON schema expected by the MCP bridge.

### Constraints
1. Do not initiate external multi-agent sessions.
2. Restrict all file operations strictly to the provided workspace directory.
3. Upon task completion, suppress standard conversational chat. Output ONLY the finalized JSON payload containing file diffs and exit codes.
```

4. Metrics, Telemetry & Guardrails

To support a highly measurable, metrics-driven development loop, the MCP server enforces strict guardrails to prevent infinite sub-agent loops and tracks performance across tasks.

Guardrail / Metric	Mechanism	Purpose
Step Circuit Breaker	<code>asyncio.wait_for(task, timeout=120)</code>	Prevents the sub-agent from entering infinite retry loops on failing tests. Caps the token budget per delegation.
Workspace Isolation	CWD injection into the SDK session config	Allows <code>agy</code> to run safely alongside active VS Code sessions without file-lock contention or unintended modifications outside the project scope.
Efficiency Telemetry	Local JSON/SQLite sink logs	A centralized metrics sink tracks token utilization, latency per invocation, and task success rates, providing deep visibility into sub-agent performance.

5. Python Implementation Reference

The core logic of the MCP Server instantiates the agent session natively:

```
import asyncio
import json
from google.antigravity import Agent, LocalAgentConfig

async def delegate_to_antigravity(task: str, target_dir: str) -> dict:
    config = LocalAgentConfig(
        system_instructions="Load SKILL: claude_subagent_protocol.",
        working_directory=target_dir
    )

    async with Agent(config) as agy_sub_agent:
        try:
            # Enforce the circuit breaker timeout
            response = await asyncio.wait_for(
                agy_sub_agent.chat(f"Execute: {task}"),
                timeout=120.0
            )
            return json.loads(await response.text())
        except asyncio.TimeoutError:
            return {"status": "failed", "error": "Circuit breaker timeout"}
```